

titanic

July 1, 2026

```
[247]: import pandas as pd
df=pd.read_csv('train.csv')
```

DATA CLEANING

```
[248]: CGECK THE NUMER OF MISISNG VALUES IN EACH COLUMN
```

```
Cell In[248], line 1
    CGECK THE NUMER OF MISISNG VALUES IN EACH COLUMN
    ~
SyntaxError: invalid syntax
```

```
[1]: df.isnull().sum()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 df.isnull().sum()

NameError: name 'df' is not defined
```

```
[250]: PERCENTAGE OF MISISNG VALUES IN EACH COLUMN
```

```
Cell In[250], line 1
    PERCENTAGE OF MISISNG VALUES IN EACH COLUMN
    ~
SyntaxError: invalid syntax
```

```
[251]: df.isnull().sum()/len(df)*100
```

```
[251]: PassengerId    0.000000
Survived           0.000000
Pclass             0.000000
```

```

Name          0.000000
Sex           0.000000
Age          19.865320
SibSp         0.000000
Parch         0.000000
Ticket        0.000000
Fare          0.000000
Cabin        77.104377
Embarked      0.224467
dtype: float64

```

cabin has most missing values almost 77 percent so we have to drop it

```
[252]: df.drop('cabin')
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[252], line 1
----> 1 df.drop('cabin')

TypeError: 'method' object is not subscriptable

```

```
[253]: df=df.drop("Cabin",axis=1)
```

```
[254]: df.head()
```

```
[254]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Embarked
0	0	A/5 21171	7.2500	S
1	0	PC 17599	71.2833	C
2	0	STON/O2. 3101282	7.9250	S
3	0	113803	53.1000	S
4	0	373450	8.0500	S

Now time to build a pipeline end to end with SimpleImputer labelencoder onehot encoding

```
[255]: df = df.drop(columns=['PassengerId', 'Name', 'Ticket'])
```

```
[256]: df.head()
```

```
[256]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

seeing index of each column

```
[257]: df.columns.tolist()
```

```
[257]: ['Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
```

Creating Transformers

```
[258]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import OneHotEncoder
```

```
[259]: Now creating another transformer for handling categorical values
```

```
Cell In[259], line 1
```

```
Now creating another transformer for handling categorical values
```

```
SyntaxError: invalid syntax
```

```
[260]: trf1=ColumnTransformer([
('impute_age',SimpleImputer(strategy="median"),[2]),
('impute_fare',SimpleImputer(strategy="median"),[5]),
('impute_embarked',SimpleImputer(strategy="most_frequent"),[6])
],remainder="passthrough")
```

```
[261]: trf2 = ColumnTransformer([
('encode_sex', OneHotEncoder(sparse_output=False), [4]),
('encode_embarked', OneHotEncoder(sparse_output=False), [2])
], remainder='passthrough')
```

```
[262]: y=df["Survived"]
X=df.drop("Survived",axis=1)
```

```
[263]: X_train,X_test,y_train,y_test=train_test_split(X,y,random_state=42,test_size=0.
↪2)
```

NOR CREATING A PIPELINE

MOST IMPORTANT DOUBT WHY DO WE NEED PIPELINE AND HOW IT AVOIDS DATA LEAKAGE

```
pipeline = Pipeline([ ('scaler', StandardScaler()), ('model', LogisticRegression())])
```

pipeline.fit(X_train, y_train) # internally it ALWAYS does: # 1. scale X_train only # 2. fit model # CANNOT mess up order!

```
[264]: pipeline=Pipeline(
    [
        ("trf1",trf1),
        ("trf2",trf2),
        ("model",DecisionTreeClassifier())
    ]
)
```

```
[265]: pipeline.fit(X_train,y_train)
prediction=pipeline.predict(X_test)
print(prediction)
```

```
[0 1 1 1 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 0 0 1
 1 1 0 0 1 0 0 1 0 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 1 1 0 0 1 0 0 0 1 1 1 1 1
 0 0 1 1 1 1 0 1 1 0 1 0 1 1 0 0 1 0 0 0 0 0 1 0 1 0 0 0 1 0 0 1 1 0 1 1 1
 0 1 1 0 0 0 0 1 0 0 1 1 1 0 0 1 1 0 0 1 0 0 1 1 0 1 0 0 0 0 1 1 0 0 1 0 1
 1 0 0 1 0 0 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 0 0 1 1]
```

```
[266]: print(trf2)

ColumnTransformer(remainder='passthrough',
                  transformers=[('encode_sex',
                                OneHotEncoder(sparse_output=False), [4]),
                                ('encode_embarked',
                                OneHotEncoder(sparse_output=False), [2])])
```

```
[268]: import numpy as np

# Input: [Pclass, Sex, Age, SibSp, Parch, Fare, Embarked]
test_input = np.array([[3, 'male', 22.0, 1, 0, 7.25, 'S']])

prediction = pipeline.predict(test_input)

if prediction[0] == 1:
    print("Survived!")
else:
```

```
print("Did not survive!")
```

Did not survive!

```

/home/owais/Desktop/reducdemo/.venv/lib/python3.14/site-
packages/sklearn/utils/validation.py:2827: UserWarning: X does not have valid
feature names, but SimpleImputer was fitted with feature names
    warnings.warn(
/home/owais/Desktop/reducdemo/.venv/lib/python3.14/site-
packages/sklearn/utils/validation.py:2827: UserWarning: X does not have valid
feature names, but SimpleImputer was fitted with feature names
    warnings.warn(
/home/owais/Desktop/reducdemo/.venv/lib/python3.14/site-
packages/sklearn/utils/validation.py:2827: UserWarning: X does not have valid
feature names, but SimpleImputer was fitted with feature names
    warnings.warn(

```

```
[269]: print("Accuracy:", pipeline.score(X_test, y_test))
```

Accuracy: 0.7821229050279329

```
[270]: from sklearn import set_config
set_config(display='diagram')

pipeline
```

```
[270]: Pipeline(steps=[('trf1',  
                        ColumnTransformer(remainder='passthrough',  
                                          transformers=[('impute_age',  
SimpleImputer(strategy='median'),  
                                                         [2]),  
                                                         ('imputer_fare',  
SimpleImputer(strategy='median'),  
                                                         [5]),  
                                                         ('impute_embarked',  
SimpleImputer(strategy='most_frequent'),  
                                                         [6])])),  
                    ('trf2',  
                     ColumnTransformer(remainder='passthrough',  
                                       transformers=[('encode_sex',  
OneHotEncoder(sparse_output=False),  
                                                         [4]),  
                                                         ('encode_embarked',  
OneHotEncoder(sparse_output=False),  
                                                         [2])])),  
                    ('model', DecisionTreeClassifier()))]
```

```
[272]: from PIL import Image
import matplotlib.pyplot as plt

img = Image.open("pipelineimp1.png")

# Increase figure size
plt.figure(figsize=(12, 12))
plt.imshow(img)
plt.axis("off")
plt.show()
```

WHY DO WE NEED PIPELINE?

And How It Avoids Data Leakage

1. WHY DO WE NEED PIPELINE?



PUTS STEPS IN THE RIGHT ORDER

Ensures preprocessing happens before model training.



AVOIDS DATA LEAKAGE

Preprocessing is learned only from **training data**, not from the **test data**.



MAKES WORKFLOW SIMPLE & CLEAN

All steps together in one object.



EASY TO REUSE & DEPLOY

The same pipeline can be used for new/unseen data.

2. WHAT IS A PIPELINE?

A **Pipeline** chains multiple steps (preprocessing + model) into one object.

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

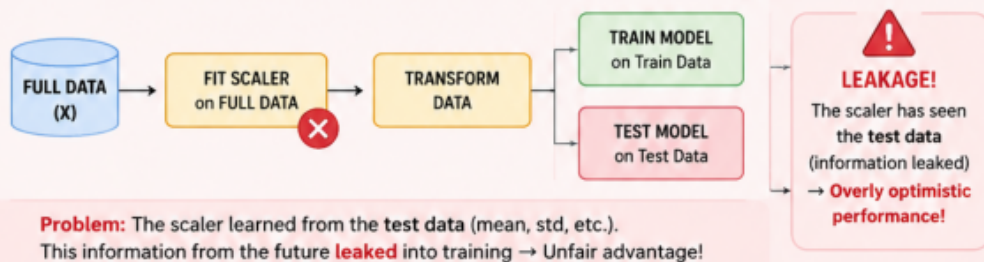
pipeline.fit(X_train, y_train)

Internally it ALWAYS does:

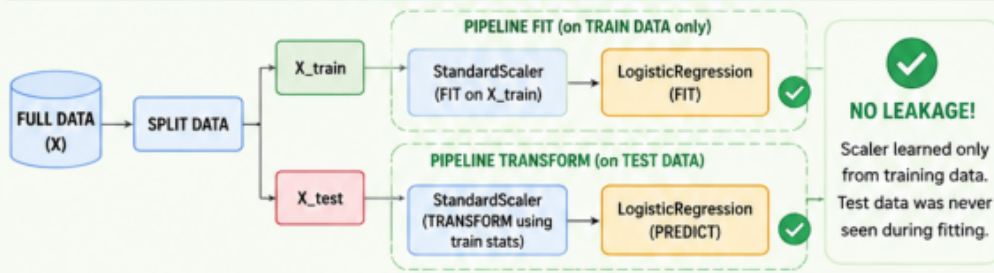
- 1 Scale **X_train** only
- 2 Fit the model
- 3 Cannot mess up order!



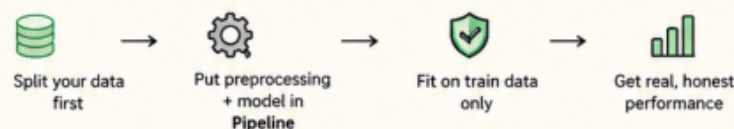
3. HOW DATA LEAKAGE HAPPENS (WITHOUT PIPELINE)



4. HOW PIPELINE PREVENTS DATA LEAKAGE



5. KEY TAKEAWAY



★ SUMMARY

Pipeline = Correct order
+ No data leakage
+ Clean, reusable code
= **Reliable ML models**



Always remember: The model should **learn patterns**, not peek at the answers!


```
[273]: from IPython.display import Image, display

display(Image(filename="pipelineimp2.png"))
```

PIPELINE vs MANUAL WORKFLOW

Same Result, But Pipeline is Smarter! 🚀

1. WITHOUT PIPELINE (MANUAL WAY)

```
# ---- TRAINING (Manual Steps) ----
array1 = trf1.fit_transform(X_train) # impute missing values
array2 = trf2.fit_transform(array1)  # encode Sex, Embarked
model.fit(array2, y_train)           # train model

# ---- TESTING (Manual Steps Again!) ----
array1 = trf1.transform(X_test)      # impute
array2 = trf2.transform(array1)      # encode
predictions = model.predict(array2)  # predict

# ---- ACCURACY (Manual) ----
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, predictions))
```

2. WITH PIPELINE (AUTOMATIC WAY)

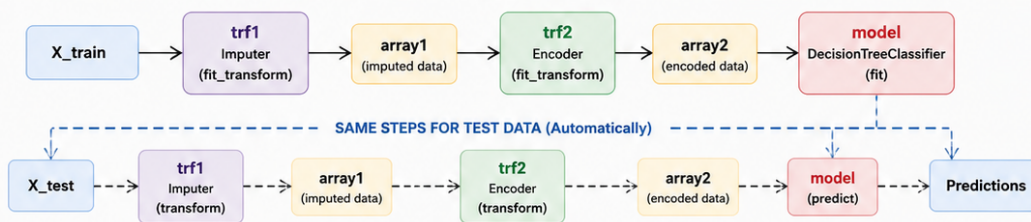
```
pipeline = Pipeline([
    ('trf1', trf1),
    ('trf2', trf2),
    ('model', DecisionTreeClassifier())
])

pipeline.fit(X_train, y_train) # does all steps automatically!
pipeline.predict(X_test)      # transforms + predicts in one line!
pipeline.score(X_test, y_test) # accuracy in one line!
```

BENEFITS OF PIPELINE

- ✓ No manual steps for test data
- ✓ Correct order of operations guaranteed
- ✓ Less code, fewer mistakes
- ✓ Avoids data leakage
- ✓ Easy to reuse and deploy

3. HOW PIPELINE WORKS INTERNALLY



4. STEP-BY-STEP COMPARISON

Step	Without Pipeline (Manual)	With Pipeline (Automatic)
1	trf1.fit_transform(X_train)	pipeline.fit(X_train, y_train) (does this)
2	trf2.fit_transform(array1)	(does this automatically)
3	model.fit(array2, y_train)	(does this automatically)
4	trf1.transform(X_test) trf2.transform(array1) model.predict(array2)	pipeline.predict(X_test) (does all transforms + predict)
5	accuracy_score(y_test, pred)	pipeline.score(X_test, y_test) (accuracy in one line)

5. KEY TAKEAWAYS

- Pipeline chains all steps together.**
Preprocessing + Model in one object.
- Prevents Data Leakage.**
Each step learns only from training data.
- Saves Time & Code.**
No need to repeat steps for test data.
- Cleaner, Safer, Smarter.**
Always use Pipeline in real ML projects! 🚀

💡 **REMEMBER:** Pipeline = Correct Order + No Data Leakage + Less Code + More Productivity ✓

[]: